



DÉVELOPPEMENT WEB

Concevoir une page web dynamique

TP4 : JavaScript



JavaScript est un langage interprété. Embarqué dans une page web ou lié à une page web, il permet d'agir sur son contenu, sa structure et sa mise en page. C'est un des trois langages web que doit connaître tout développeur web :

- ☐ HTML pour définir le contenu des pages web
- ☐ CSS pour spécifier la mise en page des pages web
- ☐ JavaScript pour programmer le comportement des pages web

L'objectif de cette activité est d'apprendre à utiliser JavaScript pour agir sur le contenu et sur le format d'une page web.



1. Objectif de l'activité :

Apprendre à utiliser JavaScript pour agir sur le contenu et sur le format d'une page web

2. JavaScript :

JavaScript est un langage de programmation à part entière. La programmation de pages Web n'est pas sa seule utilisation. De nombreux programmes et serveur utilisent JavaScript. **Node.js** est le plus connu. Certaines bases de données, comme **MongoDB** et **CouchDB**, utilisent également JavaScript comme langage de programmation.

Le saviez-vous ?

JavaScript et **Java** sont des langages totalement différents.

JavaScript a été inventé par **Brendan Eich** en 1995, et est devenu une norme ECMA en 1997.

ECMA-262 est le nom officiel de la norme. **ECMAScript** est le nom officiel du langage.

2.1. Travail à faire : Premier script

Tester le code suivant :

```
<!DOCTYPE html>
<html>
  <body>

    <h1>Mon premier script JavaScript</h1>
    <button type="button"
      onclick="document.getElementById('demo').innerHTML = 'Bonjour le
      monde !'"> Clic moi....</button>
    <p id="demo"></p>
  </body>
</html>
```

- ☐ Identifier le script et modifier le texte à afficher
- ☐ Comment le script identifie-t-il l'élément html à modifier ?
- ☐ Modifier le script pour changer le titre.

3. La syntaxe :

JavaScript emprunte la plupart des éléments de sa syntaxe à Java, C et C++ mais sa syntaxe est également influencée par Awk, Perl et Python.

- ❑ JavaScript est **sensible à la casse** et utilise l'ensemble de caractères Unicode.
- ❑ Les **instructions** (appelées *statements*) **sont séparées par des points virgules**.
- ❑ Un **bloc d'instructions est délimité par une paire d'accolades** :

```
{  
  instruction_1;  
  instruction_2;  
  .  
  .  
  .  
  instruction_n;  
}
```

- ❑ Les **noms des variables sont appelés identifiants**. Ces identifiants doivent respecter certaines règles. Un identifiant JavaScript doit commencer par **une lettre, un tiret bas (_) ou un symbole dollar (\$)**. Les caractères qui suivent peuvent être des **chiffres (0 à 9)**, des **lettres (majuscules ou minuscules)** et les **séquences d'échappement Unicode (\uXXXX)**.
- ❑ La déclaration d'une variable est précédée de :
 - **var** : La variable à une portée globale (connue dans tout le script)
 - **let** : La variable à une portée locale (connue seulement dans le bloc courant).
 - **const** : Une constante n'est connue que dans le bloc courant et n'est pas modifiable. Une constante est aussi appelée **un littéral**.
- ❑ JavaScript est un **langage à typage dynamique**. Cela signifie qu'il n'est pas nécessaire de spécifier le type de données d'une variable lors de sa déclaration. C'est lors de l'affectation que JavaScript type la donnée :

```
var variable = 42; // variable est un entier (Int)  
variable = "Merci pour le dîner."; // variable est une chaine de caractères  
x = "La réponse est "+ 42; // "La réponse est 42"  
y = 42+" est la réponse"; // "42 est la réponse"  
"37" - 7; // 30  
"37"+ 7; // "377"
```

- ❑ Les chiffres sont écrits avec ou sans décimal :

```
37 // Entier  
37.7 // Flotant
```

- ❑ Les chaînes sont du texte, écrit entre guillemets doubles ou simples :

```
"Voici une chaine de caractère"  
'Voici également une chaine de caractères'
```

- ❑ Une déclaration de fonction est construite avec le mot-clé **function**, suivi par :
 - Le nom de la fonction.
 - Une liste d'arguments à passer à la fonction, entre parenthèses et séparés par des virgules.
 - Les instructions JavaScript définissant la fonction, entre accolades, { }.

```
function carré(nombre) {  
  return nombre*nombre;  
}
```

3.1. Travail à faire : Console JavaScript :

Dans le navigateur (Mozilla Firefox), ouvrir la **Console web** (Menu → Outils de développement web → Console web)

Saisir et évaluer chacune des expressions suivantes dans la **Console web** :

```
var uneVariable = 60;
print(uneVariable);

uneVariable = "Maintenant c'est du texte...";
print(uneVariable);

x = "Des chiffres et des lettres : " + 12;
y = 12 + " : Des chiffres et des lettres";

"37" - 7;

"37"+ 7;

alert("Aïe aïe aïe!")
function carré(nombre) {
    return nombre*nombre;
}
carré(5);
```

3.2. Travail à faire : Agir sur une page

A l'aide de Sublim Text, saisir le code suivant et l'enregistrer dans un fichier html et observer le résultat dans un navigateur (Firefox).

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      h1 {
        font-size: 40px;
      }
      h2 {
        font-size: 30px;
      }
      p {
        font-size: 14px;
      }
    </style>
  </head>
  <body>

    <h1 id="h1">This is heading 1</h1>
    <h2 id="h2">This is heading 2</h2>
    <p id="p1">This is a paragraph.</p>
    <p id="p2">This is another paragraph.</p>

  </body>
</html>
```

- ☐ Ouvrir la **Console web** dans cette page
- ☐ Évaluer chacune des expressions suivantes :

```
document.getElementById("h1").innerHTML = "Bonjour le monde !"

document.getElementById("h1").style.color="red";
```

- ☐ Modifier le contenu du texte de niveau 2.
- ☐ Modifier la couleur du texte du second paragraphe en bleu.
- ☐ Modifier la couleur de l'arrière-plan du premier paragraphe en vert.

```
document.getElementById("h1").style.backgroundColor = "green";
```

- ☐ Modifier la couleur de l'arrière-plan de la page en rouge.

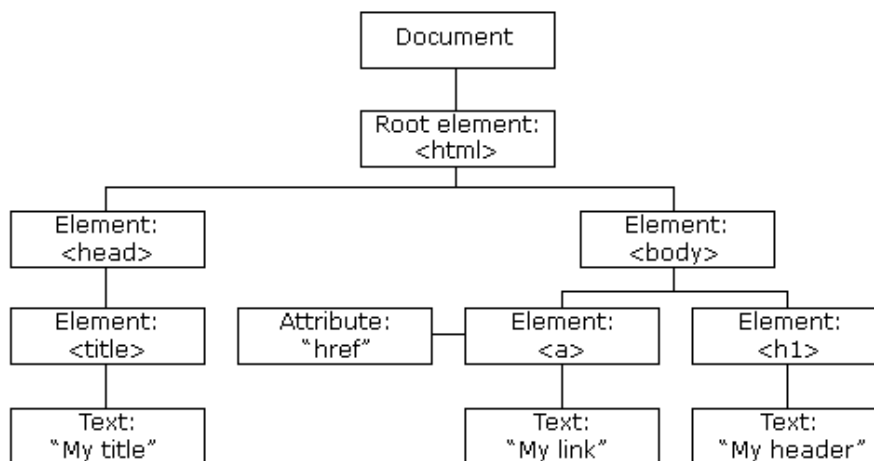
```
document.body.style.backgroundColor = "red";
```

3.3. JavaScript DOM HTML :

Lorsqu'une page Web est chargée, le navigateur crée le **Document Objet Modèle** de la page.

3.3.1. Quel est le code HTML DOM ?

Le modèle **DOM HTML** est construit comme un arbre d'objets :



Le code HTML DOM est un standard objet modèle et interface de programmation pour HTML. Il définit :

- ☐ Les éléments HTML comme des objets
- ☐ Les propriétés de tous les éléments HTML
- ☐ Les méthodes pour accéder à tous les éléments HTML

Les événements pour tous les éléments HTML_ En d'autres termes : Le code HTML DOM est une norme pour savoir comment **obtenir, modifier, ajouter ou supprimer** des éléments HTML.

Avec ce modèle objet, JavaScript obtient toute la puissance dont il a besoin pour :

- ☐ Changer tous les éléments HTML dans la page
- ☐ Changer tous les attributs HTML dans la page
- ☐ Changer tous les styles CSS dans la page
- ☐ Supprimer des éléments existants et attributs HTML
- ☐ Ajouter de nouveaux éléments et attributs HTML
- ☐ Réagir à tous les événements HTML existants dans la page
- ☐ Créer de nouveaux événements HTML dans la page

3.3.2. L'interface de programmation DOM :

Le code HTML DOM est accessible avec JavaScript (et avec d'autres langages de programmation).

Dans les DOM, tous les éléments HTML sont définis comme des objets.

L'interface de programmation est les **propriétés** et **méthodes** de chaque objet :

- **Une propriété** est une valeur que vous pouvez obtenir ou définir (comme changer le contenu d'un élément HTML).
- **Une méthode** est une action que vous pouvez faire (comme ajout ou la suppression d'un élément HTML).

L'exemple suivant modifie le contenu (innerHTML) de l'élément <p> avec id="demo":

```
<html>
  <body>

    <p id="demo"></p>

    <script>
      document.getElementById("demo").innerHTML = "Hello World!";
    </script>

  </body>
</html>
```

Dans cet exemple :

- `document` : est un objet qui contient tous les éléments de la page web
- `getElementById()` : est une méthode qui permet d'accéder à l'élément indiqué en paramètre ("demo" dans l'exemple)
- `innerHTML` : est une propriété qui contient le code HTML de l'élément accédé.

Trouver des éléments HTML

Voici quelques exemples de la façon dont vous pouvez utiliser l'objet document pour accéder et manipuler le code HTML :

Methode	Description
<code>document.getElementById(id)</code>	Trouver un élément par son identifiant id
<code>document.getElementsByTagName(name)</code>	Trouver un élément par son type de balise (tag name)
<code>document.getElementsByClassName(name)</code>	Trouver un élément par le nom de la classe (class) qui lui est affecté

- **Trouver des éléments HTML par le nom de balise HTML (Tag)**

Cet exemple trouve tous les éléments <p>:

```
var x = document.getElementsByTagName("p");
```

Essayer l'exemple sur w3schools à l'adresse suivante :

https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_getelementsbytagname2

Cet exemple trouve l'élément avec id="main" et trouve alors tous les éléments <p> qu'il contient :

```
var x = document.getElementById("main");
var y = x.getElementsByTagName("p");
```

Essayer l'exemple sur w3schools à l'adresse suivante :

https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_getelementsbytagname

- Trouver des éléments HTML par le nom de la classe qui leur sont affectés

Cet exemple retourne une liste de tous les éléments affectés par `class="intro"`.

```
var x = document.getElementsByClassName("intro");
```

Essayer l'exemple sur w3schools à l'adresse suivante :

https://www.w3schools.com/js/tryit.asp?filename=tryjs_dom_getelementsbyclassname

3.3.3. Travail à faire : Manipuler l'arbre DOM HTML :

Faire les 5 premiers exercices de w3school JS HTML DOM à l'adresse suivante :

https://www.w3schools.com/js/exercise_js.asp?filename=exercise_js_dom_html1

On souhaite ajouter dans le code HTML suivant un script qui met en gras et ajoute la balise `<blockquote>` à la citation d'Albert Einstein. Pour cela, effectuer les actions suivantes :

- Créer une variable `laCitation` qui contiendra le contenu de l'élément HTML d'id `citation`
- Ajouter lui le code HTML `<blockquote><strong>` avant et `</strong></blockquote>` après.
- Affecter le contenu de la variable `laCitation` à l'élément HTML d'id `citation`.
- Ajouter dans l'entête du fichier HTML une déclaration interne de style qui met en bleu le texte contenu dans les balises `<blockquote>`

```
<!DOCTYPE html>
<html>
  <head>

  </head>
  <body>
    <h2>Une citation</h2>
    <div id="citation">
      <p>Nous aurons le destin que nous aurons mérité.<p>
      <cite>Albert Einstein</cite>
    </div>
    <script>
      // ici le script qui met en gras et ajoute la balise <blockquote>
à la citation
    </script>

  </body>
</html>
```

Une citation

Nous aurons le destin que nous aurons mérité.

Albert Einstein

Figure 1 Résultat attendu

3.4. JavaScript DOM CSS :

Pour changer le style d'un élément HTML, utilisez la syntaxe suivante :

```
document.getElementById(id).style.property = new style
```

L'exemple suivant modifie le style d'un élément <p> :

```
<html>
  <body>

    <h1 id="h1">Hello World!</p>

    <script>
      document.getElementById("h1").style.color = "blue";
    </script>

    <p>La couleur du titre a été changée par un script.</p>

  </body>
</html>
```

3.4.1. Liste des propriétés de style CSS :

Toutes les propriétés CSS peuvent être manipulées par JavaScript. Leur nom n'est pas toujours identique à celui utilisé par CSS. Ci-dessous, une liste simplifiée des principales propriétés CSS et leur équivalent en JavaScript :

CSS	JavaScript
background	background
background-color	backgroundColor
background-image	backgroundImage
background-position	backgroundPosition
background-repeat	backgroundRepeat
border	border
border-color	borderColor
border-width	borderWidth
color	color
display	display
float	cssFloat
font	font
font-family	fontFamily
font-size	fontSize

CSS	JavaScript
font-weight	fontWeight
height	height
left	left
margin	margin
overflow	overflow
padding	padding
position	position
text-align	textAlign
text-decoration	textDecoration
text-indent	textIndent
text-transform	textTransform
top	top
vertical-align	verticalAlign
visibility	visibility

Liste complète sur w3school à l'adresse suivante : https://www.w3schools.com/jsref/dom_obj_style.asp

3.4.2. Travail à faire : Manipuler l'arbre DOM HTML :

Faire les 4 derniers exercices de w3school JS CSS DOM à l'adresse suivante :

https://www.w3schools.com/js/exercise_js.asp?filename=exercise_js_dom_html1

Compléter et tester le code suivant pour faire afficher le texte ou le faire disparaître en utilisant la propriété CSS `visibility`:

```
<!DOCTYPE html>
<html>
  <body>

    <p id="p1">Coucou me voilà ... !</p>

    <input type="button" value="Cacher"
    onclick="document.getElementById('____').____.____='hidden'">

    <input type="button" value="Afficher"
    onclick="document.getElementById('____').____.____='visible'">

  </body>
</html>
```

Compléter et tester le code suivant pour changer la couleur de fond de la page avec la couleur de la cellule du tableau est survolée :

```
<!DOCTYPE html>
<html>
<head>

</head>
<body>

<h2>Change la couleur de fond !</h2>
<p>Passe la souris sur les carrés colorés</p>

<table style="width:300px;height:100px">
  <tr>
    <td style="background-color:Khaki"
    onmouseover="document.body.____.____ = this.style.backgroundColor"
    onmouseout="document.body.____.____ = 'transparent'"
    >
    </td>
    <td style="background-color:PaleGreen"
    onmouseover="document.body.____.____ = this.style.backgroundColor"
    onmouseout="document.body.____.____ = 'transparent'"
    >
    </td>
    <td style="background-color:Silver"
    onmouseover="document.body.____.____ = this.style.backgroundColor"
    onmouseout="document.body.____.____ = 'transparent'"
    </td>
  </tr>
</table>

</body>
</html>
```

Remarque : Dans cet exercice, on a pas utilisé une méthode comme `getElementById()` pour agir sur la page web (objet `document`) mais on a agi directement sur l'élément `body` de l'objet `document`.

De la même manière, on peut agir sur d'autres éléments de la page

document.anchors	document.head
document.body	document.images
document.documentElement	document.links
document.embeds	document.scripts
document.forms	

Saisir et tester le code suivant pour ouvrir une fenêtre lorsqu'on clique sur le bouton :

```
<!DOCTYPE html>
<html>
<body>

    <p>Cliquez sur le bouton pour ouvrir une nouvelle fenêtre</p>

    <button onclick="myFunction()">Clique ici !</button>

    <script>
    function myFunction() {
        var w = window.open();
        w.document.open();
        w.document.write("<h2>Coucou !</h2>");
        w.document.close();
    }
    </script>

</body>
</html>
```

3.5. JavaScript HTML DOM Les événements :

Comme vous avez pu l'expérimenter aux travers des derniers exercices, JavaScript permet de réagir aux événements HTML.

Un JavaScript peut être exécuté lorsqu'un événement se produit, comme lorsqu'un utilisateur clique sur un élément HTML.

Pour exécuter du code lorsqu'un utilisateur clique sur un élément, ajoutez le code JavaScript à un attribut d'événement HTML :

onclick=JavaScript

Exemples d'événements HTML :

- ☐ Lorsqu'un utilisateur clique sur la souris
- ☐ Lorsqu'une page Web a chargé
- ☐ Lorsqu'une image a été chargée
- ☐ Lorsque la souris survol sur un élément
- ☐ Lorsqu'un champ d'entrée de formulaire est modifié
- ☐ Lorsqu'un formulaire HTML est soumis
- ☐ Lorsqu'un utilisateur appuie sur une touche

Dans cet exemple, le contenu de l'élément <h1> est modifié lorsqu'un utilisateur clique dessus :

```
<!DOCTYPE html>
<html>
    <body>

        <h1 onclick="this.innerHTML = 'Ooops!'">Click on this text!</h1>

    </body>
</html>
```

Essayer sur w3schools à l'adresse suivante :

https://www.w3schools.com/js/tryit.asp?filename=tryjs_event_onclick2

Remarque : Sachant que l'élément à modifier est le même que celui sur lequel a lieu l'événement, on n'a pas besoin de le rechercher avec `document.getElementById()`, on utilise `this` qui représente l'objet courant.

Vous trouverez ici la liste complète des événements HTML à l'adresse suivante :

https://www.w3schools.com/jsref/dom_obj_event.asp

3.5.1. Travail à faire : Les événements :

Télécharger sur le Moodle les images bulboff.gif et bulbon.gif les images.

Compléter et tester le code suivant pour faire afficher une ampoule allumée lorsque la souris est dessus et éteinte lorsque la souris n'est pas dessus :

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function lighton() {
      ...
    }
    function lightoff() {
      ...
    }
  </script>
</head>

<body>

  <p>Survolez l'ampoule avec le curseur...</p>

</body>
</html>
```

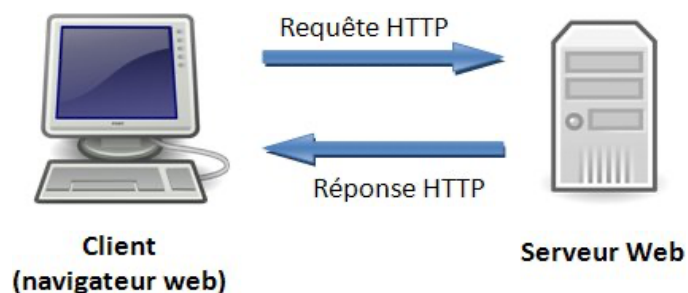
3.6. Utilisation des formulaires HTML :

Sur internet, on retrouve une configuration Client-Serveur.

- Le client demande à un serveur un service (requête HTTP)
- Le serveur lui fournit (réponse HTTP)

L'information va toujours dans ce sens.

1. Le client fait une requête HTTP au prêt d'un serveur.
2. Le serveur délivre la réponse HTTP.



En aucun cas le serveur initie la requête, pour des raisons de sécurité.

Le client peut utiliser deux méthodes pour transmettre sa requête et ses paramètres.

- Méthode **GET** : les informations sont lisibles clairement via l'URL.
- Méthode **POST** : Les informations sont cachées dans l'entête de la requête http.

3.6.1 Les formulaires HTML :

Un formulaire HTML contient des éléments de formulaire qui sont différents types d'éléments d'entrée, comme les champs de texte, des cases à cocher, des boutons radio, des boutons de soumission, et d'autres encore.

Un formulaire commence généralement par la balise `<form>` et se termine par la balise `</form>`.

3.6.2 La balise `<form>`

La balise HTML `<form>` définit un formulaire qui est utilisé pour collecter les informations d'utilisateur. Ses attributs définissent :

- ☐ L'action à exécuter lorsque le formulaire est soumis.
- ☐ Si le résultat soumis ouvrira dans un nouvel onglet de navigateur, une nouvelle fenêtre ou la fenêtre actuelle.
- ☐ La méthode HTTP (**GET** ou **POST**) à utiliser lors de l'envoi des données de formulaire

Exemple

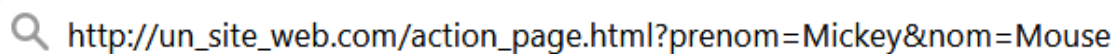
```
<form action="action_page.html" target="_blank" method="GET">
...
</form>
```

Quand utiliser GET?

La méthode par défaut lors de la soumission des données de formulaire est GET.

Toutefois, lorsque GET est utilisé, les données de formulaire soumis seront visibles dans le champ d'adresse de la page :

- ☐ Notes sur GET:



- Juxtapose form-data dans l'URL en paires nom / valeur
- La longueur d'une URL est limitée (2048 caractères)
- Ne jamais utiliser GET pour envoyer des données sensibles ! (Elles seraient visibles dans l'URL)
- Utile pour les soumissions de formulaire où un utilisateur souhaite mettre en favoris le résultat
- GET est préférable pour les données non sécurisées, comme les chaînes de requête dans Google

Quand utiliser POST?

Toujours utiliser POST si les données de formulaire contiennent des informations sensibles ou personnelles. La méthode POST ne présente pas les données de formulaire soumises dans le champ d'adresse de la page.

- ☐ Notes sur POST:
 - POST n'a pas de limite de taille, et peut être utilisé pour envoyer de grandes quantités de données.

Les soumissions de formulaire avec POST ne peuvent pas être mis en favoris

3.6.3 Entrée de formulaire :

L'élément `<input>` est l'élément le plus important d'un formulaire. L'élément `<input>` peut être affiché de plusieurs manières, en fonction de la valeur de l'attribut `type`.

Syntaxe	Description
Texte : <code><Input type = "text"></code> Texte	Définit un champ de saisie de texte d'une ligne
Choix unique : <code><Input type = "radio"></code> Choix unique : ☐	Définit un bouton radio (pour sélectionner un élément dans une liste)
Choix multiple : <code><input type="checkbox"></code> Choix multiple : ☐	Définit une case à cocher (pour sélectionner plusieurs éléments)
<code>0 <input type="range" value="50"> 100</code> 	Définit un slider
<code>a = <input type="number" value="50"></code> <code>a = </code> <input type="number" value="50"/>	Définit un nombre
Valider : <code><Input type = "submit"></code> Valide: <input type="submit" value="Envoyer"/>	Définit un bouton d'envoi (pour soumettre le formulaire)

Cliquer sur les liens suivants pour accéder à des exemples d'entrée de formulaire sur w3schools :

- [Exemple de saisie de texte](#)
- [Exemple d'Utilisation de boutons radio](#)
- [Exemple d'utilisation d'un slider](#)
- [Exemple d'utilisation d'une entrée de nombre](#)
- [Exemple de soumission d'un formulaire avec traitement côté serveur \(php\)](#)

3.6.4 Liste déroulante :

- L'élément `<select>` définit une liste déroulante.
- Les éléments `<option>` définit une option qui peut être sélectionnée.
- Par défaut, le premier élément dans la liste déroulante est sélectionné.
- Pour définir une option présélectionnée, ajoutez l'attribut `selected` à l'option
- L'attribut `multiple` permet à l'utilisateur de sélectionner plusieurs valeurs.

Exemple :

```
<select name="cars" multiple>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat" selected>Fiat</option>
  <option value="audi">Audi</option>
</select>
```

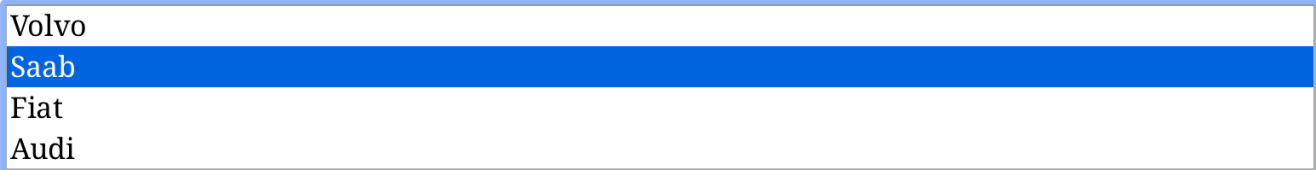


Figure 2 Résultat obtenu

Exemple d'utilisation d'une liste déroulante sur w3schools à l'adresse suivante :
https://www.w3schools.com/html/tryit.asp?filename=tryhtml_elem_select_pre

3.6.5 Zone de texte

L'élément `<textarea>` définit un champ de saisie de texte à plusieurs lignes
On peut définir la taille de la zone de texte en utilisant un style CSS :

Exemple :

```
<textarea name="message" style="width:400px; height:100px;">Le chat jouait dans le jardin.</textarea>
```

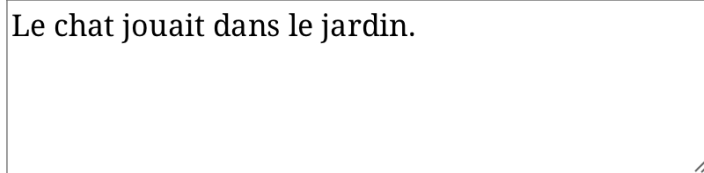


Figure 3 Résultat obtenu

Exemple d'utilisation d'une zone de texte sur w3schools à l'adresse suivante :
https://www.w3schools.com/html/tryit.asp?filename=tryhtml_elem_textarea

3.6.6 Les boutons

L'élément `<button>` définit un bouton cliquable. Une action peut lui être associée avec un script JavaScript.

Exemple

```
<button type="button" onclick="alert('Bonjour le monde!')">Clic Moi !</button>
```

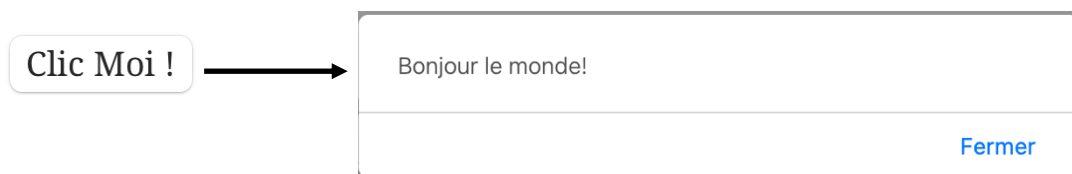


Figure 4 Résultat obtenu

Exemple d'utilisation d'un bouton cliquable sur w3schools à l'adresse suivante :
https://www.w3schools.com/html/tryit.asp?filename=tryhtml_elem_button

3.6.7 L'élément HTML5 `<datalist>` :

Les utilisateurs verront une liste déroulante des options prédéfinies dans un élément `<input>`.
L'attribut `list` de l'élément `<input>`, doit se référer à l'attribut `id` de l'élément `<datalist>`.

Exemple

```
<input list="browsers">
<datalist id="browsers">
  <option value="Internet Explorer">
  <option value="Firefox">
  <option value="Chrome">
  <option value="Opera">
  <option value="Safari">
</datalist>
```

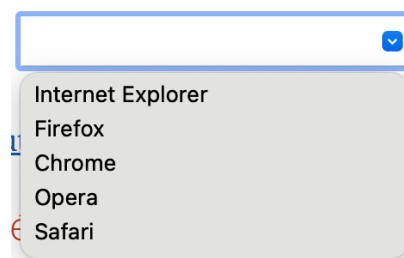


Figure 5 Résultat obtenu

Exemple d'utilisation d'une datalist sur w3schools à l'adresse suivante :
https://www.w3schools.com/html/tryit.asp?filename=tryhtml_elem_datalist

3.6.8 L'élément HTML5 <output> :

L'élément <output> représente le résultat d'un calcul opéré par un script.

Exemple

```
<form oninput="x.value=parseInt(a.value)+parseInt(b.value)">
  0
  <input type="range" id="a" name="a" value="50">
  100 +
  <input type="number" id="b" name="b" value="50">
  =
  <output name="x" for="a b"></output>
  <br><br>
  <input type="submit">
</form>
```

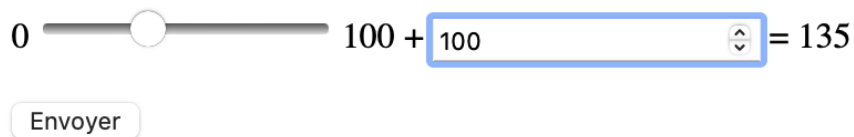


Figure 6 Résultat obtenu

3.6.9 Travail à faire : Les formulaires

Effectuer les exercices de w3school HTML Forms, Form Elements et Input Types à l'adresse suivante :

https://www.w3schools.com/html/exercise.asp?filename=exercise_html_form_attributes2

3.7. Traitement d'un formulaire avec JavaScript :

3.7.1 Premier exemple :

Saisir et tester l'exemple ci-dessous. C'est un exemple simple de traitement d'un formulaire avec JavaScript :

```
<script language="javascript">
  // Initialisation de variables
  var q1 = 0;
  var q2 = 0;
  // calcul
  function calculer () {
    score = q1 + q2;
    alert("Votre score est de : " + score + "%")
  }
</script>
<!-- la balise <hr> est affiché comme un trait horizontale par tous les
principaux navigateurs-->
<hr>
<form>
  <p>La capitale de la Suisse est :</p>
  <ol>
    <li><input type="radio" name="choice" value="1"
      onClick="q1=0">Genève</li>
    <li><input type="radio" name="choice" value="2"
      onClick="q1=0">Zurich</li>
    <li><input type="radio" name="choice" value="3"
      onClick="q1=50">Berne</li>
  </ol>
  <p>Le Pô se trouve en :</p>
  <ol>
    <li><input type="radio" name="choice2" value="1"
      onClick="q2=50">Italie</li>
```

```
<li><input type="radio" name="choice2" value="2"
onClick="q2=0">Allemagne</li>
<li><input type="radio" name="choice2" value="3"
onClick="q2=0">Espagne</li>
</ol>
<input type="button" name="ProcessButton" value="Voir le résultat!"
onClick="calculer()" ">
</form>
<hr>
```

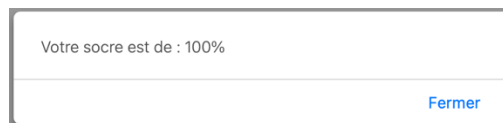
La capitale de la Suisse est :

1. ☐ Genève
2. ☐ Zurich
3. ☒ Berne

Le Pô se trouve en :

1. ☒ Italie
2. ☐ Allemagne
3. ☐ Espagne

Voir le résultat!



Explications :

- ☐ Le script contient l'initialisation de deux variables globales `q1` et `q2` et la définition d'une fonction.
- ☐ La fonction `calculer()` affiche une alerte qui contient le résultat de l'opération `q1+q2`
- ☐ Lorsque l'utilisateur clique sur un bouton radio, `q1` ou `q2` est affecté.
- ☐ Lorsque l'utilisateur clique sur le bouton, la fonction `calculer()` est exécutée.

3.7.2 Deuxième exemple :

Saisir et tester l'exemple ci-dessous. Dans cet exemple, les code HTML et JavaScript sont strictement séparés.

```
<script language="javascript">
function init () {
    document.getElementById("quiz").onsubmit = calculer2;
}

function calculer2() {

    // f est un raccourci qui évite de réécrire document.forms["quiz"]
    var f = document.forms["quiz"];

    // score = 0 au début
    var score = 0;

    for (var i=0; i < (f.length); i++) {
        // On boucle sur tous les éléments du formulaire
        // Si l'élément est coché on ajoute sa valeur
        if (f.elements[i].checked) {
            score = score + parseInt(f.elements[i].value);
        }
    }
    // Affichage du résultat
    alert ("Votre score est de : " + score + "%");
}
window.onload = init;
</script>

<form id="quiz" action="#" method="get"> <br/>
    <p>La capitale de la Suisse est :</p>
    <ol>
        <li><input type="radio" name="choice" value="0">Genève</li>
        <li><input type="radio" name="choice" value="0">Zurich</li>
        <li><input type="radio" name="choice" value="50">Berne</li>
    </ol>

    <p>Le Pô se trouve en :</p>
    <ol>
        <li><input type="radio" name="choice2" value="50">Italie</li>
        <li><input type="radio" name="choice2" value="0">Allemagne</li>
        <li><input type="radio" name="choice2" value="0">Espagne</li>
    </ol>
    <input type="submit" value="Voir le résultat!" />

</form>
```

Explications :

- Il n'y a plus de code JavaScript dans le Html. La fonction `init()` est exécutée au chargement de la page : `window.onload = init;`
- La fonction `init()` exécute la fonction `calculer2()` si le formulaire 'quizz' a été soumis.
- La fonction `calculer2()` boucle sur la longueur du formulaire (scrute tous ses éléments) et ajoute la valeur (attribut `value`) de chaque élément coché. La fonction `calculer2()` affiche une alerte qui contient le score lorsqu'au moins un élément est sélectionné (variable `score` existe).

3.7.2 Travail à faire : Traitement d'un formulaire avec JavaScript

Télécharger sur le Moddle l'archive **Questionnaire_Parcoursup.zip** et la décompresser sur le bureau.

Le but de cet exercice est de compléter les fichiers HTML, CSS et JavaScript pour réaliser une page web contenant un formulaire dont le traitement sera exécuté par un script JavaScript.

Figure 7 Résultat attendu

La page HTML contient un formulaire pour :

- ☐ Saisir le prénom
- ☐ Saisir le nom
- ☐ Sélectionner une classe :
 - **Tale pro SN**
 - **Tale STI2D**
 - **Tale Générale**
- ☐ Sélectionner les deux spécialités que vous garderez en terminale :
 - **BUT GEII**
 - **BUT Réseaux-Télécoms**
 - **BUT Informatique**
 - **BTS SIO**
 - **CPGE**
 - **BTS CIEL IR**
 - **BTS CIEL ER**
 - **LICENCE**
- ☐ Un bouton pour soumettre le formulaire
- ☐ Un élément `<h1 id="resultat">` qui contiendra le résultat du traitement et permettra son affichage.

Le fichier CSS contient les éléments de styles pour la mise en forme du HTML

Le fichier JavaScript traite le formulaire lors de sa soumission. Il contient :

- ☐ Une fonction `init()`, exécutée au chargement de la page. Cette fonction :
 - Exécute la fonction `resultat()` lors de la soumission du formulaire
 - Modifie l'élément `id="resultat"` comme suit lors du chargement de la page :
 - Si au moins un des deux vœux = **BTS CIEL IR** alors afficher "**Bon choix !**" sinon afficher "**Mauvais choix !**"
- ☐ Une fonction `resultat()` qui :
 - Vérifie le remplissage correct du formulaire (Nom et Prénom et Classe et 2 vœux d'orientation)
 - Affiche une alerte en conséquence
 - Affiche une alerte correspondant au résultat

3.7.3 Travail à faire : Calculatrice simple :

Télécharger sur le Moddle l'archive **calculatrice.zip** et la décompresser sur le bureau.

En vous inspirant des travaux précédents, compléter les fichiers HTML, CSS et Javascript afin de réaliser la page web suivante proposant une calculatrice simple :

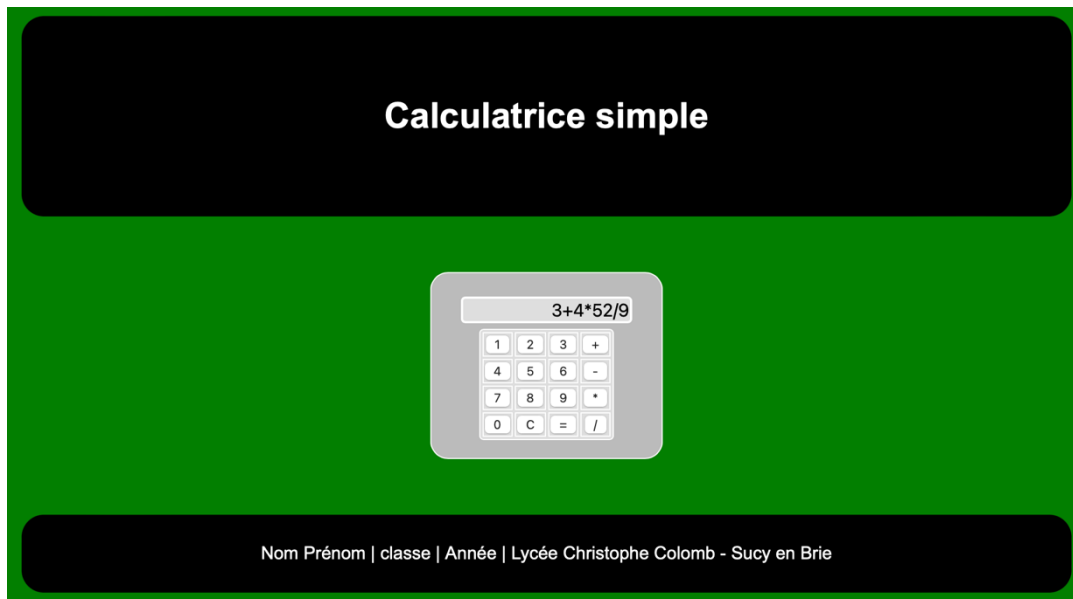


Figure 8 Résultat attendu

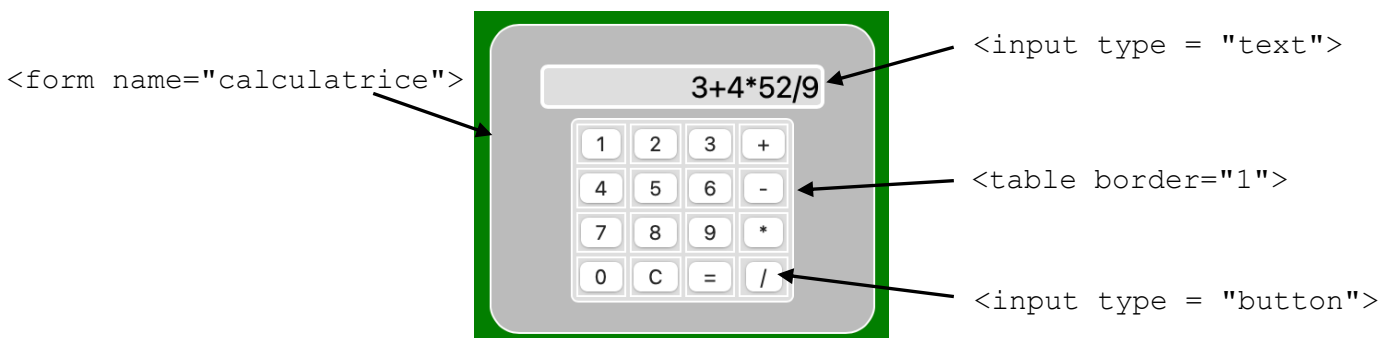


Figure 9 structure HTML de la calculatrice

La page HTML contient un formulaire pour :

- ☐ Saisir, grâce à des boutons, les chiffres et les opérateurs du calcul à réaliser.
- ☐ Un bouton « = » pour effectuer et afficher le calcul.
- ☐ Un « écran » permet d'afficher la saisie et le résultat du calcul
- ☐ Un bouton « C » pour effacer l'écran
- ☐ L'ensemble des boutons sont contenu dans un tableau HTML.

Le fichier CSS contient les éléments de styles pour la mise en forme du HTML

Le fichier JavaScript contient :

- ☐ Une fonction `calculer()`, exécutée lors du clic sur le bouton « = ». Cette fonction :
Mémorise dans une variable, la chaîne de caractères afficher sur l'écran.
Exécute la fonction javascript `eval()` pour réaliser le calcul.
Affiche le résultat du calcul sur l'écran.
- ☐ Une fonction `afficher()`, exécutée lors du clic sur les boutons « 0 » à « 9 » et « = », « - », « * » et « / ». Cette fonction affiche à l'écran les chiffres et opérateurs pressés.
- ☐ Une fonction `effacer()`, exécutée lors du clic sur le boutons « C ». Cette fonction efface à l'écran.

4. Aller plus loin...

Il est fortement conseillé d'effectuer les exercices auto-corrigés du site de référence w3school.com :

- <https://www.w3schools.com/js/default.asp> : Rappels de cours et tutoriels complets
- https://www.w3schools.com/js/js_exercises.asp : Exercices auto-corrigés